



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

There slides are mainly extracted from:

Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021

Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024

Software architecture and design 3

Francesco Guerra
francesco.guerra@unimore.it

Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017

The Clean Architecture

Software Architect

- ▶ **Software architects** guide teams toward a design that **maximizes productivity**.
- ▶ **Architecture** defines the **shape of a system** through:
 - ▶ **Division into components**
 - ▶ **Arrangement of components**
 - ▶ **Communication between components**
- ▶ **Purpose:** to **facilitate development, deployment, operation, and maintenance**.
- ▶ **Strategy:** **keep options open** for as long as possible.
- ▶ **Main goal:** support the **system life cycle**, not just make it work.
- ▶ **Good architecture** makes the system **easy to understand, develop, maintain, and deploy**.
- ▶ **Ultimate goal:** **minimize lifetime cost and maximize programmer productivity**.

Software Architect

▶ DEVELOPMENT

- ▶ A software that is **hard to develop** is not likely to have a long and **healthy** lifetime.

▶ DEPLOYMENT

- ▶ To be effective, a software must be deployable. **The higher the cost of deployment, the less useful the system is.**

▶ OPERATION

- ▶ The architecture has a limited impact on system operations.
 - ▶ Almost **any operational difficulty can be resolved by throwing more hardware without impacting the software architecture.**
- ▶ The system architecture makes the operation of readily apparent to the developers.
 - ▶ The system architecture should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers.

Software Architect

▶ MAINTENANCE

- ▶ The never-ending parade of new features and the trail of defects and corrections consume vast amounts of human resources.
- ▶ The primary cost of maintenance is in spelunking and risk.
 - ▶ **Spelunking is the cost of digging through the software, trying to determine the best place and strategy to add a new feature or to repair a defect.**
 - ▶ While making such changes, the likelihood of creating inadvertent defects is always there, adding to the cost of risk.
- ▶ The architecture should mitigates these costs.
 - ▶ By separating the system into components, and isolating those components through stable interfaces, it is possible to reduce the risk of inadvertent breakage.

Keeping options open

- ▶ **Software value:** consists of **behavior** and **structure**.
- ▶ **Structure** allows software to **quickly and easily change machine behavior**.
- ▶ **Flexibility** depends on:
 - ▶ **System shape**
 - ▶ **Arrangement of components**
 - ▶ **Connections between components**
- ▶ Keep software **soft** by leaving as many **options (details that don't matter)** open as long as possible.
- ▶ **Software decomposition:**
 - ▶ **Policy:** embodies **business rules and procedures**.
 - ▶ **Details:** elements needed for humans, systems, and programmers to interact with the policy (e.g., IO, databases, web systems, servers, frameworks, protocols) but **do not affect policy behavior**.
- ▶ Goal: **highlight policy** as essential, while making **details irrelevant**.
- ▶ A **good architect maximizes undecided choices**, keeping flexibility high.

Keeping options open

- ▶ Early in development, it's **not necessary to choose a database**.
 - ▶ The **high-level policy** should not depend on whether it's relational, distributed, NoSQL, or flat files.
- ▶ Early **web server choice** is also unnecessary.
 - ▶ The policy shouldn't know it's delivered over the web or depend on **HTML, AJAX, JSP, JSF**.
- ▶ Developing the **high-level policy without committing to details** allows:
 - ▶ **Delaying decisions** until more information is available.
 - ▶ **Better-informed choices** about technologies.
 - ▶ **Flexibility to experiment** with different options.

Leaving option open

- ▶ A **good architecture** balances all concerns with a **component structure** that satisfies them mutually.
- ▶ Achieving this balance is **very challenging**.
- ▶ Often, we don't fully know:
 - ▶ All use cases
 - ▶ Operational constraints
 - ▶ Team structure
 - ▶ Deployment requirements
- ▶ Even if known, these factors **change over the system's life cycle**.
- ▶ In short, architectural goals are **indistinct and inconstant**.

Decoupling layers

- ▶ **Architect's challenge:** support all use cases without knowing all of them.
- ▶ Knows the **basic intent** of the system.
- ▶ Use **Single Responsibility Principle (SRP)** and **Common Closure Principle (CCP)** to:
 - ▶ **Separate things that change for different reasons**
 - ▶ **Collect things that change for the same reasons**
- ▶ **Examples of different reasons for change:**
 - ▶ **UI changes** are independent of **business rules** → separate the UI from business logic.
 - ▶ **Business rules:**
 - ▶ **Application-specific:** e.g., input field validation → tied to the application.
 - ▶ **Domain-specific:** e.g., interest calculation → tied to the domain.
 - ▶ **These rules change at different rates**, so they should be separated.
- ▶ **Technical details** (database, query language, schema) **do not affect business rules or UI.**
- ▶ **Recommended separation:**
 - ▶ **UI layer / Application-specific business rules / Application-independent business rules / Database layer**

Decoupling use cases

- ▶ **Use cases change for different reasons**
 - ▶ Example: adding an order changes differently than deleting an order.
- ▶ **Use cases are a natural way to divide the system**
 - ▶ They are **vertical slices** cutting through the **horizontal layers** of the system.
- ▶ Each use case may involve:
 - ▶ **UI**
 - ▶ **Application-specific business rules**
 - ▶ **Application-independent business rules**
 - ▶ **Database functionality**
- ▶ **Pattern insight:**
 - ▶ Decoupling elements that change for different reasons **lets you add new use cases without interfering with old ones.**
 - ▶ Grouping **UI and database support for each use case** reduces the chance that new use cases affect older ones.

Decoupling mode

- ▶ **Decoupling components** improves system operations.
 - ▶ When **use cases** are isolated, those requiring **high throughput** can be managed separately from those with lower demands.
 - ▶ By separating the **user interface** and the **database** from the **business rules**, they can run on **different servers**.
 - ▶ Components that require **higher bandwidth** can be **replicated across multiple servers** to improve scalability.
- ▶ To truly benefit from this separation, the decoupling must be **implemented properly**:
 - ▶ Modules should be **independent services** that **communicate over a network**.
- ▶ Such components are often referred to as **services** or **microservices**, depending on their size and complexity.
 - ▶ An architecture based on this approach is known as a **Service-Oriented Architecture (SOA)**.
- ▶ The **mode of decoupling** is an **architectural choice** that helps keep **future options open** for system evolution.

Duplication

- ▶ **Duplication** is usually undesirable in software systems.
 - ▶ **True duplication:** every change in one instance requires the same change in all its copies.
 - ▶ **False (accidental) duplication:** two similar pieces of code evolve differently – they change for **different reasons** or at **different rates**.
- ▶ **Vertical Separation (Use Cases)**
 - ▶ When separating **use cases vertically**, you may be tempted to **couple them** because they share:
 - ▶ similar structures
 - ▶ similar algorithms
 - ▶ similar database queries or schemas
- ▶ Always **verify that the duplication is real** before trying to merge or reuse code.

Duplication

- ▶ **Horizontal Separation (Layers)**
 - ▶ When separating **layers horizontally**, you may notice that:
 - ▶ a **database record** has a structure very similar to a **screen view**.
 - ▶ It may seem easier to pass the record **directly to the UI**,
 - ▶ but this increases **coupling** between layers.
 - ▶ Instead, **create a separate view model and copy the relevant data**.
 - ▶ This preserves **proper decoupling** between database and presentation layers.

Levels of Decoupling

1. Source Level

- ▶ Control dependencies between **source code modules** so that changes in one module **don't require recompilation** of others.
- ▶ All components run in the **same address space** and communicate through **function calls**.
- ▶ The result is a **single executable**, often called a **monolithic structure**.

2. Deployment Level

- ▶ Control dependencies between **deployable units** (e.g., .jar files, DLLs, shared libraries).
- ▶ Changes in one module's source **don't require rebuilding or redeploying** others.
- ▶ Components may:
 - ▶ still live in the **same address space** and use **function calls**, or
 - ▶ live in **different processes** on the same machine and communicate via **IPC, sockets, or shared memory**.
- ▶ Key idea: components are **independently deployable units**.

Levels of Decoupling

3. Service Level

- ▶ Dependencies are reduced to **data-level communication**.
- ▶ Components interact **only through network packets**.
- ▶ Each service is **fully independent** – both in **execution** and in **source/binary evolution**.
- ▶ Typical examples: **services** or **microservices** architectures.

Choosing the Right Decoupling Mode

- ▶ Early in a project, it's hard to know the best decoupling mode.
- ▶ As the system grows, the **optimal mode** may change:
 - ▶ A single-server system might later need **separate servers, deployable units, or services**.
- ▶ **Service-level decoupling** by default is **expensive**:
 - ▶ More development effort
 - ▶ Higher resource usage
 - ▶ Unnecessary boundaries if not needed
- ▶ A good architecture allows:
 - ▶ **Start as a monolith** (single file deployment)
 - ▶ **Grow into deployable units**
 - ▶ **Expand into independent services or microservices**
- ▶ It should also allow **reversing** the progression if needed.
- ▶ **Key principle:** leave the **decoupling mode flexible**, so **small deployments** can remain monolithic, while **large deployments** can scale with services.

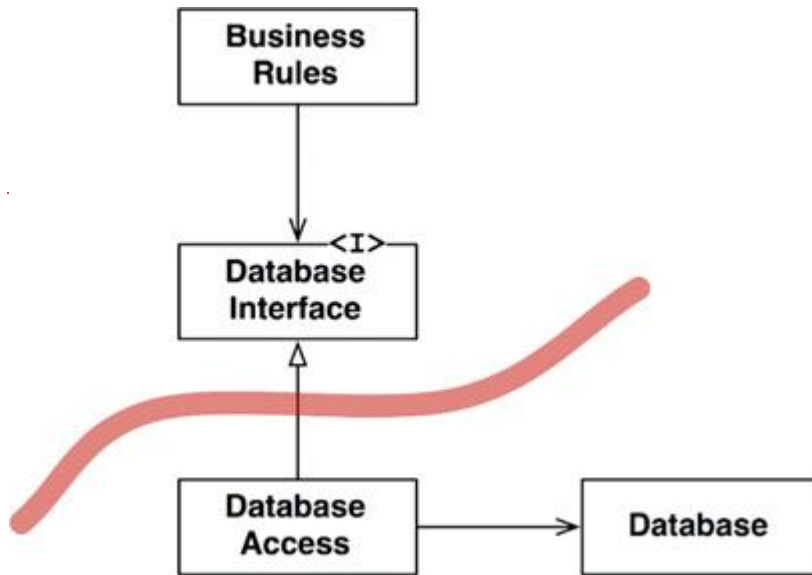
Boundaries: drawing lines

- ▶ **Architecture is the art of drawing boundaries** between software elements.
- ▶ Boundaries **restrict elements** on one side from knowing or depending on those on the other.
- ▶ **Goal:** minimize human effort required to **build and maintain** the system.
- ▶ **Coupling**, especially from **premature decisions**, undermines this goal.
 - ▶ **premature decisions** = decisions unrelated to **business requirements/use cases**, e.g.:
 - ▶ Frameworks
 - ▶ Databases
 - ▶ Web servers
 - ▶ Utility libraries
 - ▶ Dependency injection
- ▶ **A good architecture** defers these decisions until the **latest possible moment**, **without major impact** on the system.

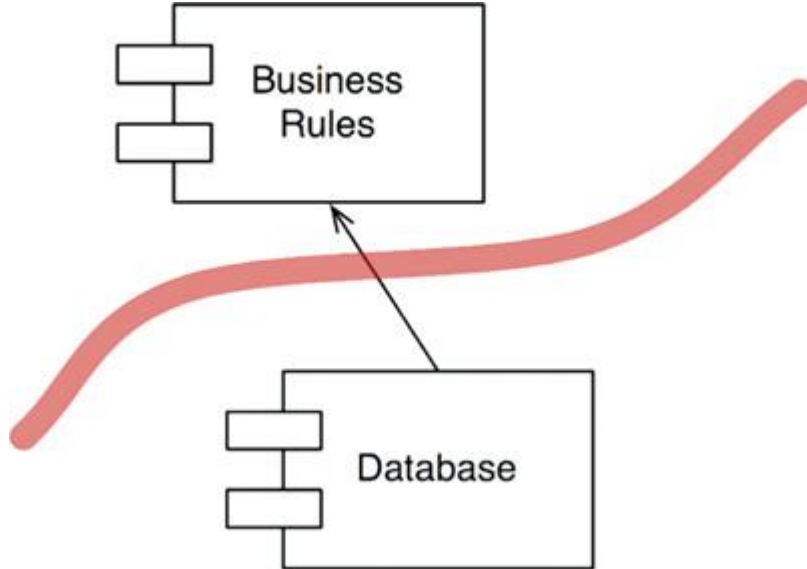
Boundaries: drawing lines

- ▶ You draw lines between things that matter and things that don't.
 - ▶ The GUI doesn't matter to the business rules, so there should be a line between them.
 - ▶ The database doesn't matter to the GUI, so there should be a line between them.
 - ▶ The database doesn't matter to the business rules, so there should be a line between them.

Boundaries: drawing lines

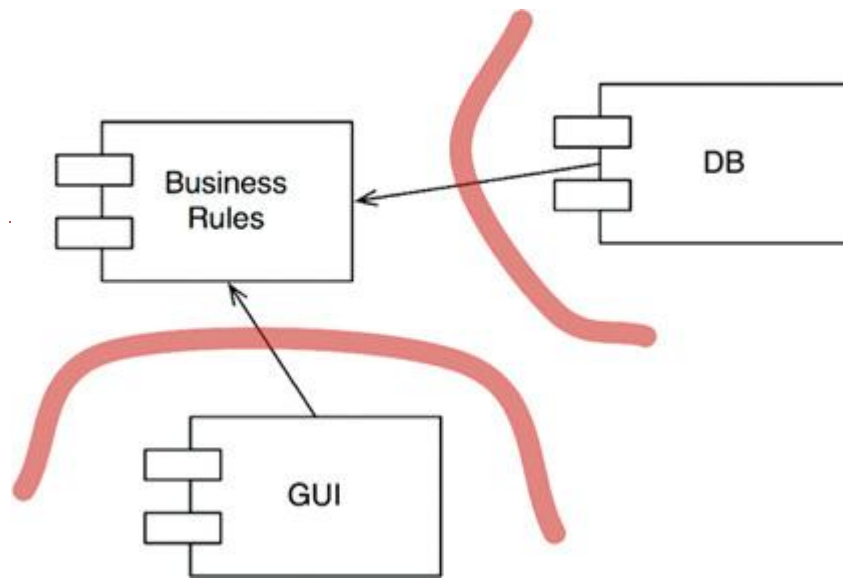


- ▶ The database is a tool that the business rules can use *indirectly*.
 - ▶ The business rules don't need to know about the schema, the query language, or any other detail about the database.
- ▶ The business rules need to know that there is a set of functions that can be used to fetch or save data.
- ▶ This allows us to put the database behind an interface.
 - ▶ Note the two arrows leaving the DatabaseAccess class.
 - ▶ Those two arrows point away from the DatabaseAccess class. That means that none of these classes knows that the DatabaseAccess class exists.



Boundaries: drawing lines

- ▶ At the component level,
 - ▶ The Database knows about the BusinessRules.
 - ▶ The Database does not matter to the BusinessRules, but the Database cannot exist without the BusinessRules
 - ▶ The BusinessRules do not know about the Database.
 - ▶ This implies that
 - ▶ the DatabaseInterface classes live in the BusinessRules component,
 - ▶ the DatabaseAccess classes live in the Database component.
- ▶ With this architecture the BusinessRules could use any kind of database.
 - ▶ The Database component could be replaced with many different implementations—the BusinessRules don't care.



Plugin Architecture

- ▶ The history of software development technology is the story of how to conveniently create plugins to establish a scalable and maintainable system architecture.
 - ▶ The core business rules are kept separate from, and independent of, those components that are either optional or that can be implemented in many different forms
 - ▶ Because the user interface in this design is considered to be a plugin, we have made it possible to plug in many different kinds of user interfaces.
 - ▶ They could be web based, client/server based, SOA based, Console based, or based on any other kind of user interface technology.
 - ▶ The same is true of the database.

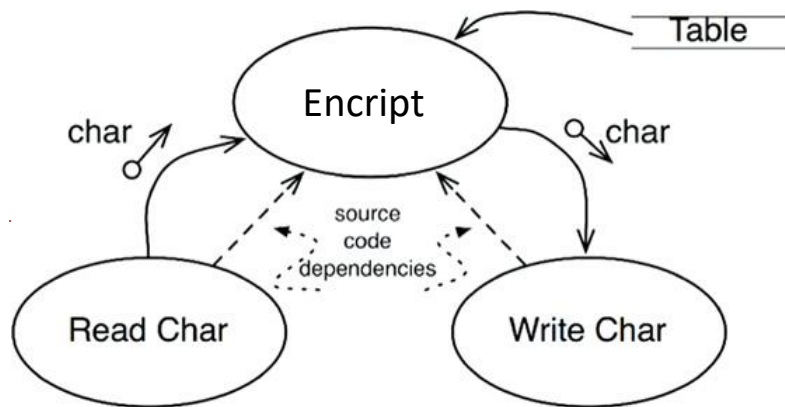
Boundary Anatomy

▶ Boundary Crossing

- ▶ At runtime, a boundary crossing is nothing more than a function on one side of the boundary calling a function on the other side and passing along some data.
 - ▶ One source code module changes, other source code modules may have to be changed or recompiled, and then redeployed.
 - ▶ Managing and building firewalls against this change is what boundaries are all about.

Policy and Level

- ▶ A **program describes the policy** for transforming inputs into outputs.
- ▶ Policies can be broken down into smaller statements for:
 - ▶ Business rules calculations
 - ▶ Report formatting
 - ▶ Input validation
- ▶ **Architecture design** groups policies **based on how they change**:
 - ▶ Policies that change for the same reasons **belong together** in the same component.
- ▶ Components form a **directed acyclic graph (DAG)**:
 - ▶ **Nodes**: components containing policies at the same level
 - ▶ **Edges**: dependencies between components at different levels
- ▶ **Dependencies** are mostly **source code, compile-time dependencies**.
- ▶ **Good architecture principle**: low-level components depend on high-level components.



Level

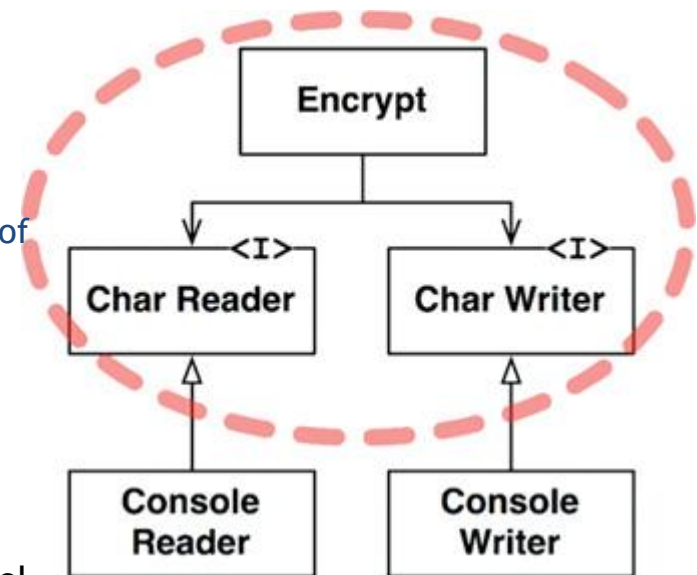
- ▶ **Level definition:** distance from inputs and outputs.
 - ▶ Policies **farther from I/O** → **higher-level**.
 - ▶ Policies managing I/O → **lowest-level**.
- ▶ **Example:** simple encryption program
 - ▶ Reads characters from input / Translates using a table / Writes characters to output
- ▶ **Data flows:** shown as **curved solid arrows**.
- ▶ **Source code dependencies:** shown as **straight dashed lines**.
- ▶ **Encrypt component:** highest-level, farthest from I/O.
- ▶ **Key insight:**
 - ▶ **Data flow** $\neq$ **source code dependencies**
 - ▶ Good architecture **decouples source dependencies from data flow**, aligning them with **level**.

```
function encrypt() {  
    while(true)  
        writeChar(translate(readChar()));  
}
```

Level

► The above is incorrect architecture:

- the high-level encrypt function depends on the lower-level readChar and writeChar functions.
- A better architecture is shown in the class diagram that decouples the high-level encryption policy from the lower-level input/output policies.
 - This makes the encryption policy usable in a wide range of contexts.
 - Higher-level policies tend to change less frequently, and for more important reasons, than lower-level policies.
 - Lower-level policies tend to change frequently, and with more urgency, but for less important reasons.
- Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level policies, reduces the impact of change.
 - Trivial but urgent changes at the lowest levels of the system have little or no impact on the higher, more important levels.



Business Rules

- ▶ By dividing an application into business rules and plugins, we see what business rules actually are.
- ▶ **Business rules:** rules or procedures that **make or save the business money**.
 - ▶ Exist independently of computers.
 - ▶ Example: A bank charging N% interest is a business rule, whether computed by a program or not.
- ▶ **Critical Business Rules**
 - ▶ Essential to the business itself, exist **even without a computer system**.
 - ▶ Often require **Critical Business Data**, which also exists independently of automation.
- ▶ **Entities**
 - ▶ Critical rules and their data are **inextricably bound**.
 - ▶ These combined objects are called **Entities** in software design.

Entities

- ▶ **Entity:** an object that embodies **critical business rules** operating on **critical business data**.
- ▶ Contains or has **easy access** to the critical data.
- ▶ **Interface:** functions implementing the **critical business rules**.
- ▶ **Example:** Loan entity
 - ▶ Contains 3 pieces of critical business data
 - ▶ Exposes 3 related critical business rules
- ▶ **Purpose:**
 - ▶ Groups software that implements a **concept critical to the business**.
 - ▶ Separates it from other concerns like **databases, UI, or frameworks**.
 - ▶ **Independent** of presentation or storage; purely business logic.
- ▶ **Key principle:**
 - ▶ Bind **critical data** and **critical rules** together in a **single, separate module**.

Loan
- principle - rate - period
+ makePayment() + applyInterest() + chargeLateFee()

Use cases

- ▶ **Use Cases vs Entities**
- ▶ **Not all business rules are pure Entities.**
 - ▶ Some rules define **how the automated system operates.**
 - ▶ These rules **do not exist in a manual system**; they make sense only in software.
- ▶ **Use Case:** a description of how an automated system is used.
 - ▶ **Specifies:**
 - ▶ **Input** provided by the user
 - ▶ **Output** returned to the user
 - ▶ **Processing steps** to produce the output
- ▶ **Distinction:**
 - ▶ **Entities** → critical business rules + data, independent of automation
 - ▶ **Use cases** → application-specific rules, tied to how the system is used

Gather Contact Info for New Loan

Input: Name, Address, Birthdate, D.L. #, SSN, etc.
Output: Same info for readback + credit score.

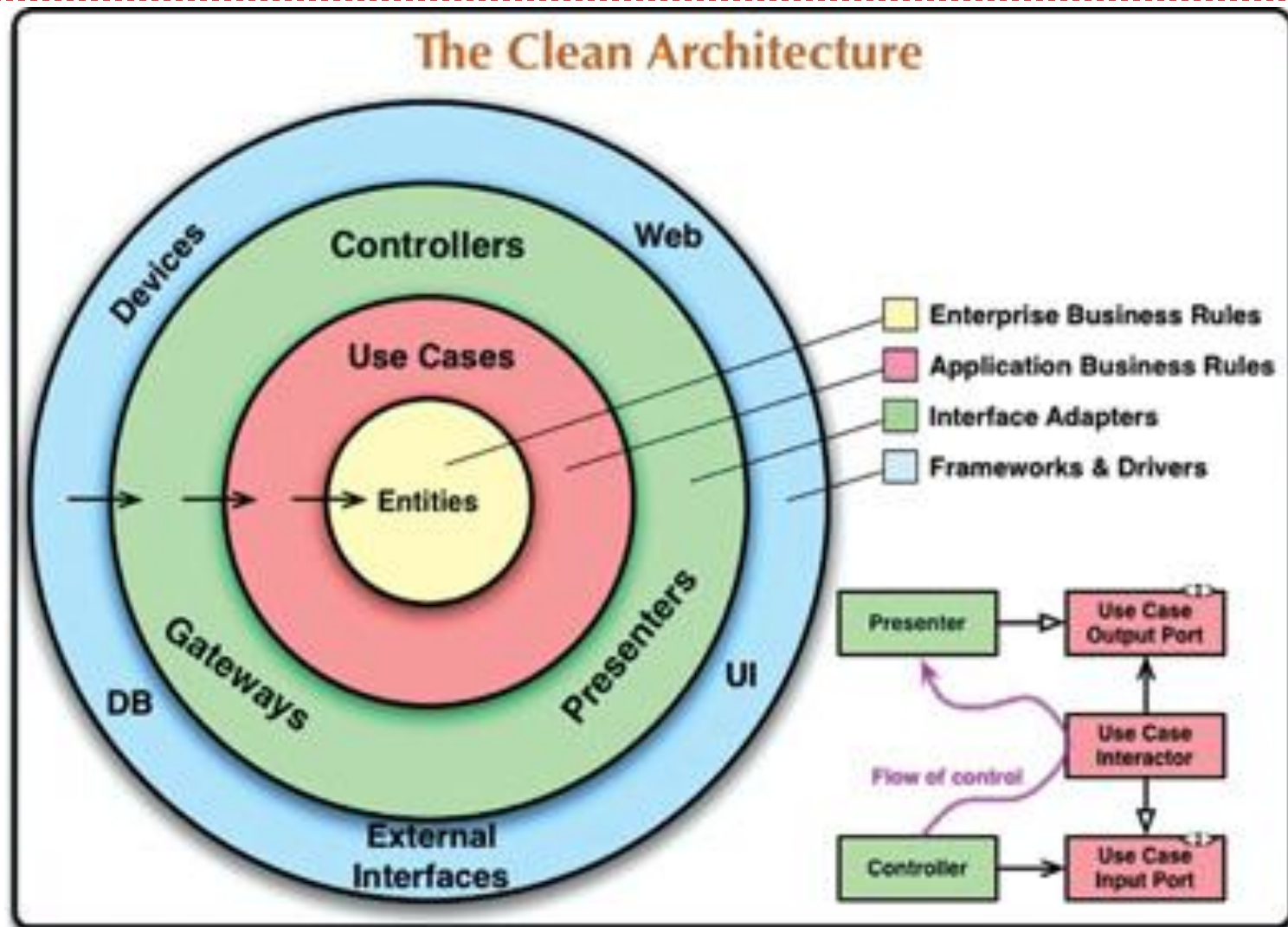
Primary Course:

1. Accept and validate name.
2. Validate address, birthdate, D.L.#, SSN, etc.
3. Get credit score.
4. If credit score is < 500 activate Denial.
5. Else create Customer and activate Loan Estimation.

Use cases

- ▶ **Use cases specify how and when** critical business rules in Entities are invoked.
- ▶ They are **application-specific rules** governing interactions between users and Entities.
- ▶ **Do not describe:**
 - ▶ User interface (web, console, service)
 - ▶ How data enters or leaves the system
- ▶ **Key relationships:**
 - ▶ Entities have **no knowledge** of use cases
 - ▶ Use cases know about **Entities**
 - ▶ High-level concepts (Entities) are **independent**
 - ▶ Low-level concepts (Use cases) are **closer to inputs/outputs**
- ▶ **Dependency rule:**
 - ▶ **Use cases depend on Entities**
 - ▶ **Entities do not depend on use cases**

The Clean Architecture



Architecture features

- ▶ *Independent of frameworks.*
The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
- ▶ *Testable.* The business rules can be tested without the UI, database, web server, or any other external element.
- ▶ *Independent of the UI.* The UI can change easily, without changing the rest of the system. A web UI could be replaced with a console UI, for example, without changing the business rules.
- ▶ *Independent of the database.* Your business rules are not bound to the database.
- ▶ *Independent of any external agency.* In fact, your business rules don't know anything at all about the interfaces to the outside world.

The Clean Architecture

- ▶ **Concentric circles** represent different areas of software:
 - ▶ **Inner circles:** policies (high-level, core business rules)
 - ▶ **Outer circles:** mechanisms (low-level, implementation details)
- ▶ **Dependency Rule:**
 - ▶ Source code dependencies **must point inward** toward higher-level policies
 - ▶ **Inner circles** cannot know anything about **outer circles**
 - ▶ Outer circles may impact inner circles, but not vice versa
- ▶ **Entities (inner circle):**
 - ▶ Encapsulate **enterprise-wide Critical Business Rules**
 - ▶ Reusable across multiple applications
- ▶ **Use cases layer (middle circle):**
 - ▶ Contains **application-specific rules**
 - ▶ Orchestrates data flow to/from Entities
 - ▶ Directs Entities to execute **Critical Business Rules**
 - ▶ Changes in this layer **do not affect Entities**
 - ▶ Also isolated from external details like **UI or database**

The Clean Architecture

- ▶ **Interface Adapters Layer:**
 - ▶ Converts data **between** use cases/entities and external systems (e.g., database, web).
 - ▶ Contains the **MVC components** of a GUI:
 - ▶ Models = simple data structures
 - ▶ Controllers/Presenters = orchestrate flow to/from use cases
 - ▶ **Inner layers must not know anything about the database or external systems**
 - ▶ Also handles other adapters for **external services**
- ▶ **Frameworks & Tools Layer (Outermost):**
 - ▶ Contains **databases, web frameworks, and other tools**
 - ▶ Minimal code: mostly **glue code** connecting to the interface adapters
 - ▶ **All details** of external systems reside here
- ▶ **Key principle:**
 - ▶ The **web and database are just details**, not part of core business logic

The Clean Architecture

- ▶ **Dependency Rule:**

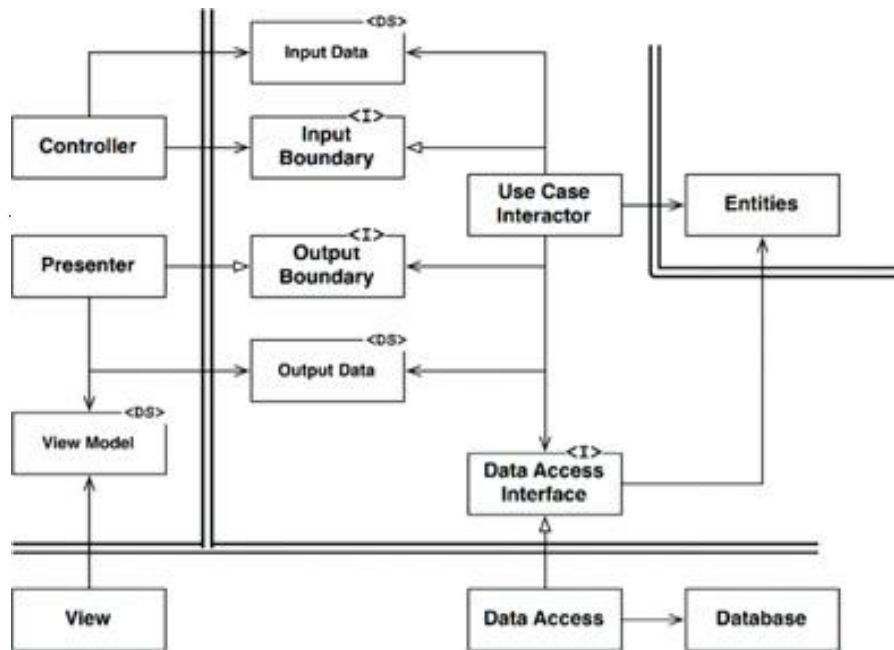
- ▶ Source code dependencies **always point inward** toward higher-level policies.

- ▶ **Flow of control:**

- ▶ Controllers → Use Cases → Presenters
- ▶ **Control flow** moves outward, but **dependencies point inward**

- ▶ **Dependency Inversion Principle:**

- ▶ Interfaces and inheritance **invert dependencies** across boundaries
- ▶ **Example:**
 - ▶ Use Case needs to call Presenter
 - ▶ **Cannot call directly** (violates Dependency Rule)
 - ▶ **Solution:** Use an **interface in the inner circle**, implemented by the Presenter in the outer circle



A typical scenario

A typical scenario for a web-based Java system using a database.

- ▶ **UseCaseInteractor → Entities**
 - ▶ Interprets input data
 - ▶ Controls Entities
 - ▶ Uses **DataAccessInterface** to load data from the database
- ▶ **UseCaseInteractor → OutputBoundary → Presenter**
 - ▶ Gathers data from Entities
 - ▶ Constructs **OutputData**
 - ▶ Passes **OutputData** through the **OutputBoundary**
- ▶ **Presenter → ViewModel → View**
 - ▶ Repackages **OutputData** into **ViewModel**
 - ▶ View displays data in the HTML page

The main component

- ▶ The Main component is the ultimate detail—the lowest-level policy. It is the initial entry point of the system.
 - ▶ Nothing, other than the operating system, depends on it.
 - ▶ Its job is to create all the Factories, Strategies, and other global facilities, and then hand control over to the high-level abstract portions of the system.
- ▶ It is in this Main component that dependencies should be injected by a Dependency Injection framework.
 - ▶ Once they are injected into Main, Main should distribute those dependencies normally, without using the framework.
- ▶ Think of Main as the dirtiest of all the dirty components.

Documenting the Software architecture

4 + 1 view model of software architecture

- ▶ A logical view, which shows the key abstractions in the system as objects or object classes.
- ▶ A process view, which shows how, at run-time, the system is composed of interacting processes.
- ▶ A development view, which shows how the software is decomposed for development.
- ▶ A physical view, which shows the system hardware and how software components are distributed across the processors in the system.
- ▶ Related using use cases or scenarios (+1)

The C4 Model

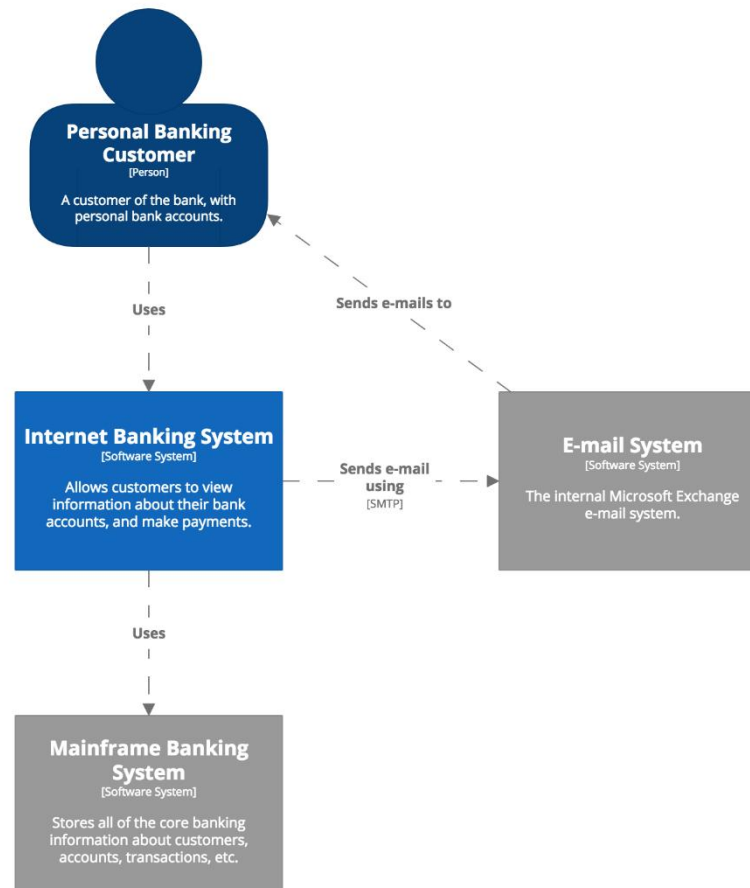
1. Level 1: System Context diagram
2. Level 2: Container diagram
3. Level 3: Component diagram
4. Level 4: Code



Level 1: System Context diagram

- ▶ A System Context diagram is a good starting point for diagramming and documenting a software system, allowing you to step back and see the big picture.
 - ▶ Draw a diagram showing your system as a box in the centre, surrounded by its users and the other systems that it interacts with.
- ▶ Detail isn't important here as this is your zoomed out view showing a big picture of the system landscape. The focus should be on people (actors, roles, personas, etc) and software systems rather than technologies, protocols and other low-level details. It's the sort of diagram that you could show to non-technical people.
- ▶ **Scope:** A single software system.
- ▶ **Primary elements:** The software system in scope.
Supporting elements: People and software systems directly connected to the software system in scope.
- ▶ **Intended audience:** Everybody, both technical and non-technical people, inside and outside of the software development team.

Level 1: System Context diagram



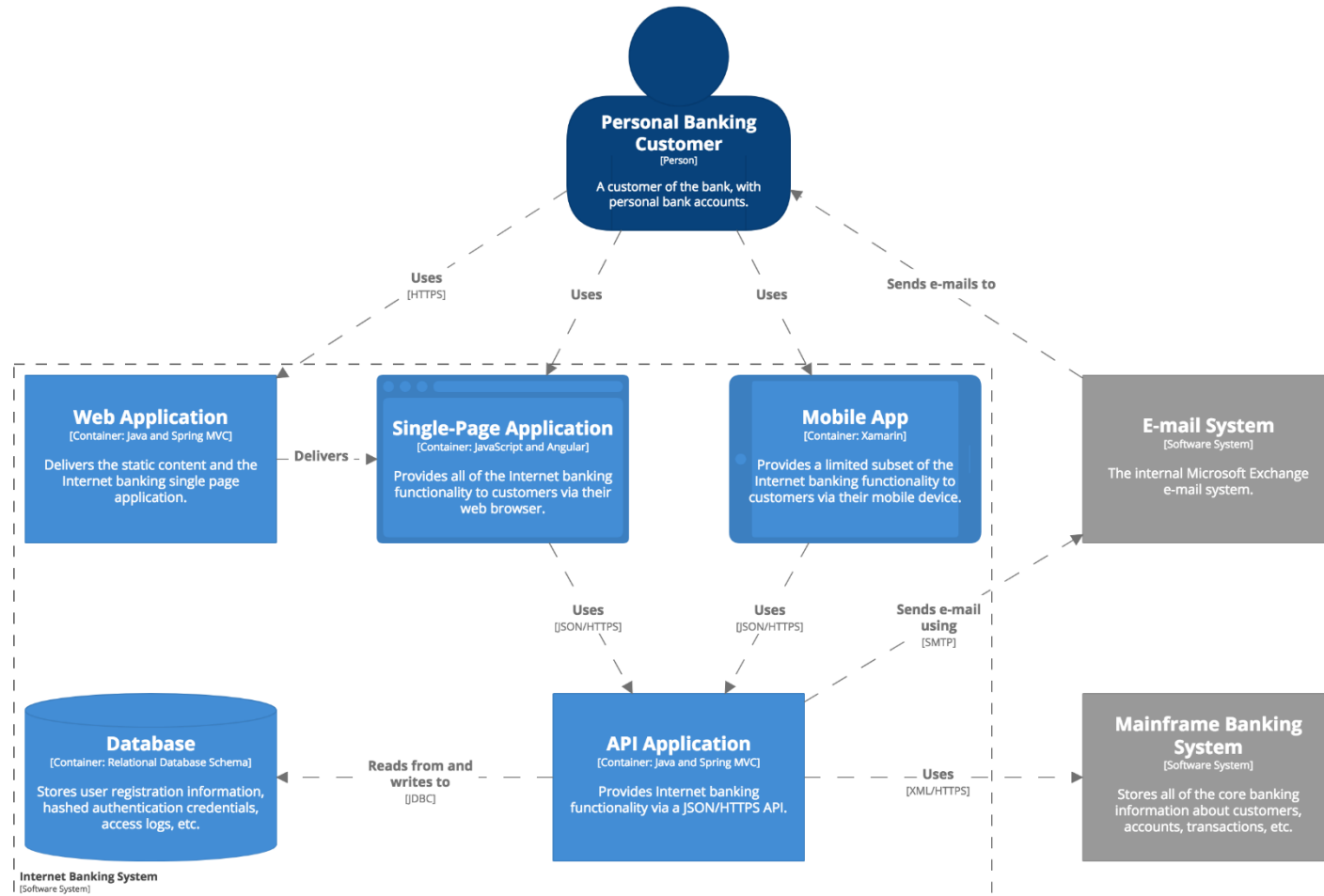
System Context diagram for Internet Banking System

The system context diagram for the Internet Banking System.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 2: Container diagram

- ▶ Once you understand how your system fits in to the overall IT environment, a really useful next step is to zoom-in to the system boundary with a Container diagram.
 - ▶ A "container" is something like a server-side web application, single-page application, desktop application, mobile app, database schema, file system, etc. Essentially, a container is a separately runnable/deployable unit (e.g. a separate process space) that executes code or stores data.
 - ▶ The Container diagram shows the high-level shape of the software architecture and how responsibilities are distributed across it. It also shows the major technology choices and how the containers communicate with one another.
 - ▶ It's a simple, high-level technology focussed diagram that is useful for software developers and support/operations staff alike.
- ▶ **Scope:** A single software system.
- ▶ **Primary elements:** Containers within the software system in scope.
Supporting elements: People and software systems directly connected to the containers.
- ▶ **Intended audience:** Technical people inside and outside of the software development team; including software architects, developers and operations/support staff.
- ▶ **Notes:** This diagram says nothing about deployment scenarios, clustering, replication, failover, etc.

Level 2: Container diagram



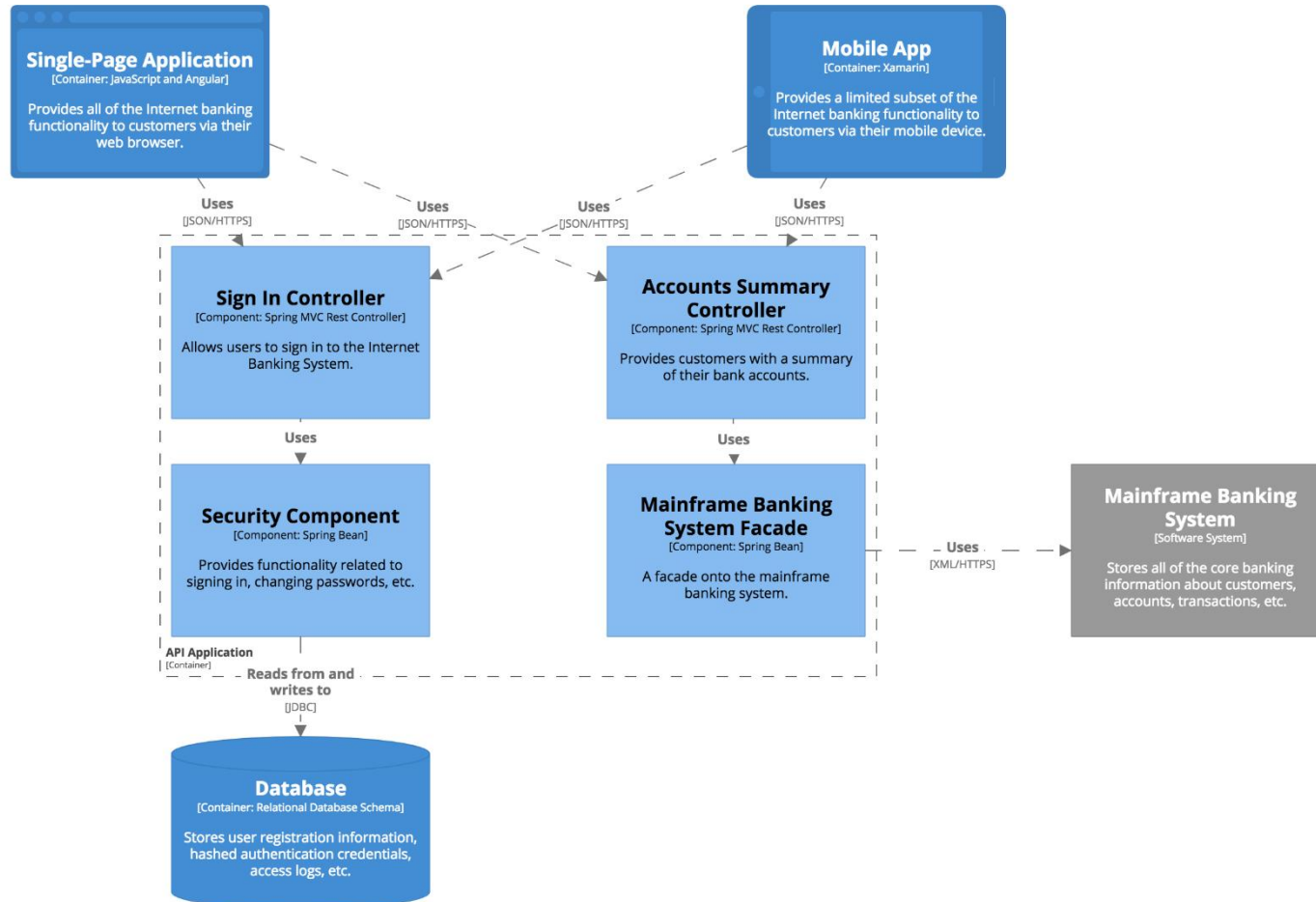
Container diagram for Internet Banking System

The container diagram for the Internet Banking System.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 3: Component diagram

- ▶ Next you can zoom in and decompose each container further to identify the major structural building blocks and their interactions.
- ▶ The Component diagram shows how a container is made up of a number of "components", what each of those components are, their responsibilities and the technology/implementation details.
- ▶ **Scope:** A single container.
- ▶ **Primary elements:** Components within the container in scope.
Supporting elements: Containers (within the software system in scope) plus people and software systems directly connected to the components.
- ▶ **Intended audience:** Software architects and developers.

Level 3: Component diagram



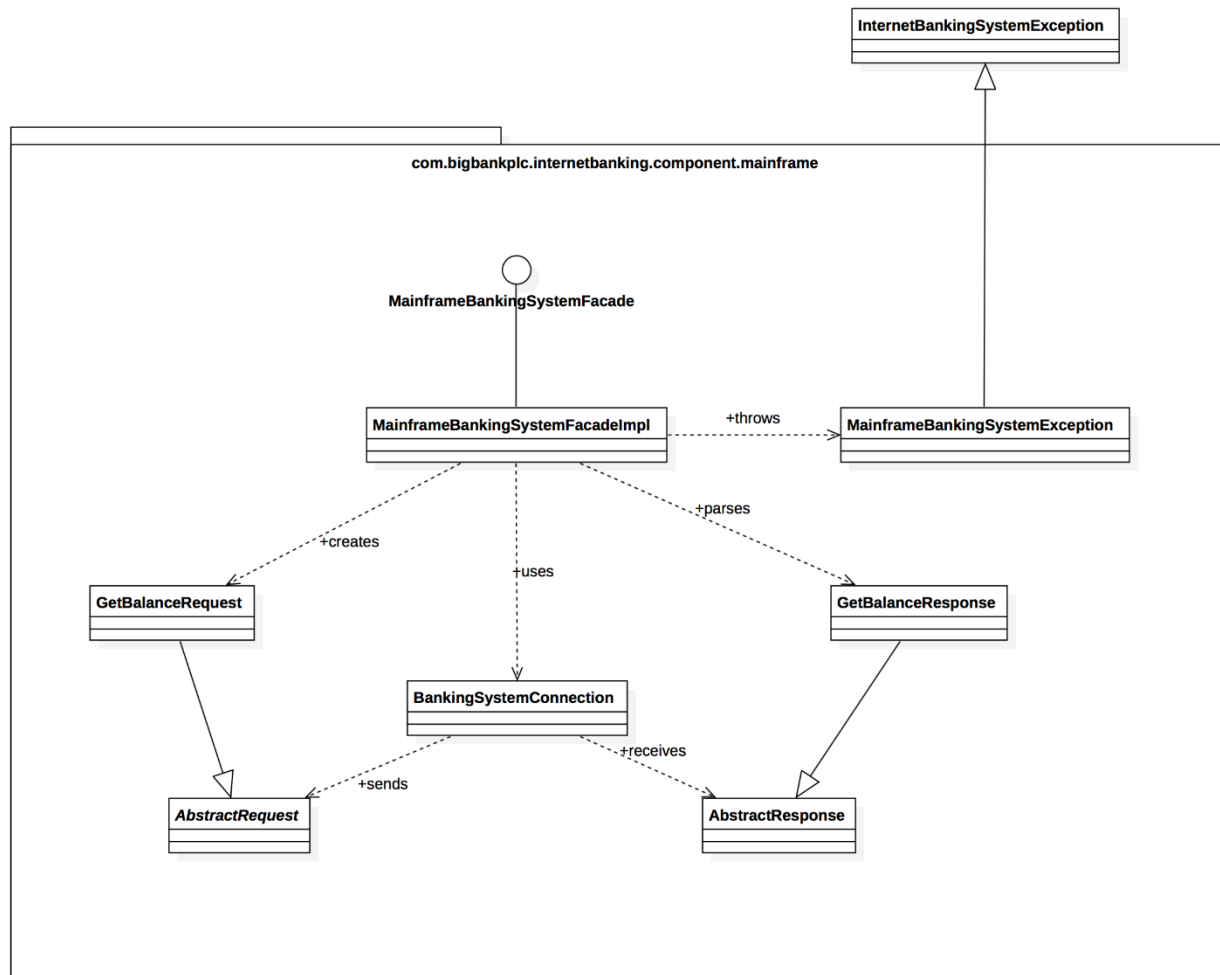
Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 4: Code

- ▶ Finally, you can zoom in to each component to show how it is implemented as code; using UML class diagrams, entity relationship diagrams or similar.
- ▶ This is an optional level of detail and is often available on-demand from tooling such as IDEs. Ideally this diagram would be automatically generated using tooling (e.g. an IDE or UML modelling tool), and you should consider showing only those attributes and methods that allow you to tell the story that you want to tell. This level of detail is not recommended for anything but the most important or complex components.
- ▶ **Scope:** A single component.
- ▶ **Primary elements:** Code elements (e.g. classes, interfaces, objects, functions, database tables, etc) within the component in scope.
- ▶ **Intended audience:** Software architects and developers.

Level 4: Code





References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
- ▶ Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017
- ▶ Sommerville, Software engineering 9, Pearson
- ▶ Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021
- ▶ Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024
- ▶ Software Architecture Fundamentals Understanding the Basics
- ▶ Neal Ford, Mark Richards: Fundamentals, Patterns, AntiPatterns, Soft Skills, Continuous Delivery, and Code Analysis Tools, O'Reilly Media March 2014
- ▶ Rachel Roumeliotis: Variations in event-driven architecture
<http://radar.oreilly.com/2015/02/variations-in-event-driven-architecture.html>
- ▶ <https://martinfowler.com/articles/serverless.html>

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.